QuillAudits

AUDIT REPORT

May 2026

For

atomyx

Table of Content

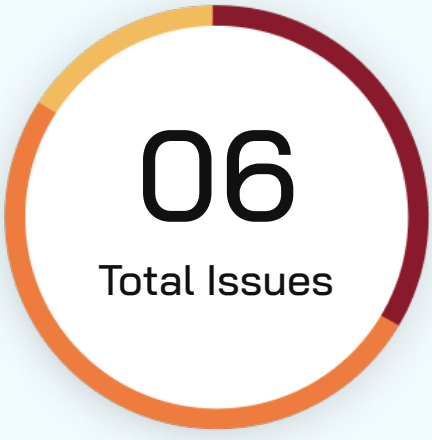
Executive Summary

Project Name	Atomyx
Project URL	https://atomyx.capital/
Overview	Solana program implements a Token-2022 transfer hook that enforces whitelist-based transfer restrictions at the protocol level. It initializes PDAs for configuration and dynamically resolves whitelist entries for sender and receiver using extra account metadata. Each wallet has a dedicated WhitelistEntry PDA storing its whitelist status, managed by an authorized controller defined in WhitelistConfig. During every token transfer, the execute hook validates that both sender and receiver are actively whitelisted, otherwise the transaction fails. The program supports single and batch whitelist updates, with strict PDA verification for integrity. A fallback handler ensures compatibility with the Token-2022 hook interface by routing CPI calls correctly.
Audit Scope	The scope of this Audit was to analyze the atomyx Smart Contracts for quality, security, and correctness.
Source Code link	solana/transfer-hook/programs/transfer-hook/src/lib.rs
Contracts in Scope	lib.rs
Branch	Master
Commit Hash	fc20ebe1ee991077983f96d28778c9a111a40d2d
Language	Rust
Blockchain	Solana
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	4th May 2026 - 11th May 2026
Updated Code Received	20th May 2026
Review 2	21st May 2026
Fixed In	dec5559396b139f0b258f1829c18c8ef2c4e35ad

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	2 (33.3%)
High	0 (0.0%)
Medium	3 (50.0%)
Low	1 (16.7%)
Informational	0 (0.0%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	0	0
Partially Resolved	0	0	0	0	0
Resolved	**2**	0	**3**	**1**	0

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Attacker can bypass whitelist using valid entries from other whitelisted addresses	Critical	Resolved
2	Attacker can use whitelist config from one mint with token accounts from different mint	Critical	Resolved
3	Anyone can initialize the whitelist hook for any mint and set themselves as authority	Medium	Resolved
4	Attacker can pre-fund the ExtraAccountMetas PDA to prevent initialization	Medium	Resolved
5	Batch whitelist function does not create accounts if they don't exist, silently skipping uninitialized entries	Medium	Resolved
6	Execute function does not verify token accounts are in transferring state	Low	Resolved

Checked Vulnerabilities

We have scanned the solana program for commonly known and more specific vulnerabilities. Here are some of the commonly known vulnerabilities that we considered:

- Signer authorization
- Account data matching
- Sysvar address checking
- Owner checks
- Type cosplay
- Initialization
- Arbitrary cpi
- Duplicate mutable accounts
- Bump seed canonicalization
- PDA Sharing
- Incorrect closing accounts
- Missing rent exemption checks
- Arithmetic overflows/underflows
- Numerical precision errors
- Solana account confusions
- Casting truncation
- Insufficient SPL token account verification
- Signed invocation of unverified programs

Techniques and Methods

Throughout the audit of Solana Programs, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Critical Severity Issues

Attacker can bypass whitelist using valid entries from other whitelisted addresses

Resolved

Path

lib.rs

Function Name

`execute()`

Description

The `execute()` function validates that both source and destination whitelist entries have `is_active = true`, but it does not verify that these entries actually belong to the token account owners. This allows an attacker to pass valid whitelist entries belonging to different whitelisted wallets.

Vulnerable Code

```rust
pub fn execute(ctx: Context<Execute>, _amount: u64) -> Result<()> {
    let source_whitelist = &ctx.accounts.source_whitelist_entry;
    let destination_whitelist = &ctx.accounts.destination_whitelist_entry;


    // Only checks is_active, NOT wallet ownership
    require!(
        source_whitelist.is_active,
        TransferHookError::SenderNotAllowed
    );
    require!(
        destination_whitelist.is_active,
        TransferHookError::ReceiverNotAllowed
    );
    Ok(())
}
```

Impact

• Complete bypass of whitelist: Any non-whitelisted address can execute transfers using other whitelisted addresses' entries
• Loss of asset control: The whitelist mechanism becomes ineffective

Recommendation

Extract the wallet address from each token account data and verify it matches the corresponding whitelist entry:

```
    let source_whitelist = &ctx.accounts.source_whitelist_entry;
    let destination_whitelist = &ctx.accounts.destination_whitelist_entry;


    // Parse token account data to extract owner (bytes 32-64)
    let source_owner = Pubkey::new(&source_account.data[32..64]);
    let dest_owner = Pubkey::new(&destination_account.data[32..64]);


    // Verify whitelist entries belong to actual token owners
    require!(
        source_whitelist.wallet == source_owner,
        TransferHookError::InvalidSourceEntry
    );
    require!(
        destination_whitelist.wallet == dest_owner,
        TransferHookError::InvalidDestinationEntry
    );


    require!(
        source_whitelist.is_active,
        TransferHookError::SenderNotAllowed
    );
    require!(
        destination_whitelist.is_active,
        TransferHookError::ReceiverNotAllowed
    );
    Ok(())
}
```

Add error code:

```
```rust
#[error_code]
pub enum TransferHookError {
    #[msg("Source whitelist entry does not match token account owner")]
    InvalidSourceEntry,
    #[msg("Destination whitelist entry does not match token account owner")]
    InvalidDestinationEntry,
    // … existing errors
}
```
```

Attacker can use whitelist config from one mint with token accounts from different mint

Resolved

Path

lib.rs

Function Name

`execute()`

Description

The `execute()` function receives multiple accounts but never verifies that all accounts belong to the **same mint**. An attacker can pass:

- Whitelist config and entries for Mint-A
- Token accounts (source/destination) for Mint-B

Since the code only checks PDAs and `is_active` status, it doesn't validate mint consistency. Since different mints can be passed, the whitlisting becomes inaffective because an attacker can pass an entry for mint A while passing config & mint accounts as mint B.

Vulnerable Code

```
// Execute receives these accounts:
pub struct Execute<'info> {
    pub source: UncheckedAccount<'info>,                // Could be from Mint-B
    pub mint: UncheckedAccount<'info>,                  // Mint-A (passed freely)
    pub destination: UncheckedAccount<'info>,           // Could be from Mint-B
    // ...
    pub whitelist_config: Account<'info, WhitelistConfig>, // Points to Mint-A
}


// No validation that:
// source.mint == mint
// destination.mint == mint
// whitelist_config.mint == mint
```

Remediation

Validate that the mint stored in all relevant accounts matches:

```
pub fn execute(ctx: Context<Execute>, _amount: u64) -> Result<()> {
    let source_account = &ctx.accounts.source;
    let destination_account = &ctx.accounts.destination;
    let mint_account = &ctx.accounts.mint;
    let whitelist_config = &ctx.accounts.whitelist_config;
    let source_whitelist = &ctx.accounts.source_whitelist_entry;
    let destination_whitelist = &ctx.accounts.destination_whitelist_entry;


    let mint_key = mint_account.key();
```

```
    // Verify all mints match
    // Token account layout: mint(0..32), owner(32..64), …
    let source_mint = Pubkey::new(&source_account.data[0..32]);
    let dest_mint = Pubkey::new(&destination_account.data[0..32]);


    require!(
        source_mint == mint_key,
        TransferHookError::SourceMintMismatch
    );
    require!(
        dest_mint == mint_key,
        TransferHookError::DestinationMintMismatch
    );
    require!(
        whitelist_config.mint == mint_key,
        TransferHookError::ConfigMintMismatch
    );
    require!(
        source_whitelist.mint == mint_key,
        TransferHookError::SourceEntryMintMismatch
    );
    require!(
        destination_whitelist.mint == mint_key,
        TransferHookError::DestinationEntryMintMismatch
    );


    // Then proceed with whitelist checks
    require!(source_whitelist.is_active, TransferHookError::SenderNotAllowed);
    require!(destination_whitelist.is_active,
TransferHookError::ReceiverNotAllowed);


    Ok(())
}
```

Add error code:

```rust
#[error_code]
pub enum TransferHookError {
    #[msg("Source token account mint does not match")]
    SourceMintMismatch,
    #[msg("Destination token account mint does not match")]
    DestinationMintMismatch,
    #[msg("Whitelist config mint does not match")]
    ConfigMintMismatch,
    #[msg("Source whitelist entry mint does not match")]
    SourceEntryMintMismatch,
    #[msg("Destination whitelist entry mint does not match")]
    DestinationEntryMintMismatch,
    // … existing errors
}
```

Medium Severity Issues

Anyone can initialize the whitelist hook for any mint and set themselves as authority

Resolved

Path

lib.rs

Function Name

`initialize()`

Description

The `initialize()` function accepts any signer as the authority and does not verify they are the mint authority stored in the mint account. This allows any attacker to initialize a transfer hook for any mint and gain full control over whitelist management.

Vulnerable Code

```rust
pub fn initialize(ctx: Context<Initialize>) -> Result<()> {
    // Only checks that authority is a signer, NOT that they're mint authority
    let config = &mut ctx.accounts.whitelist_config;
    config.mint = mint.key();
    config.authority = ctx.accounts.authority.key();  // Sets whoever called it
first
    Ok(())
}



#[derive(Accounts)]
pub struct Initialize<'info> {
    #[account(mut)]
    pub authority: Signer<'info>,  // Any signer accepted
    pub mint: UncheckedAccount<'info>,
    // ...
}
```

Impact

Unauthorized control: Attackers can take over whitelist management of any mint

Recommendation

Extract and verify the mint authority from the mint account before accepting initialize, make sure only mint authority of a mint can call initialize.

Attacker can pre-fund the ExtraAccountMetas PDA to prevent initialization

Resolved

Path

lib.rs

Function Name

`initialize()`

Description

The `initialize()` function derives a PDA address for `extra_account_metas` and then uses `system_program::create_account()` to create it. An attacker can pre-fund this predictable PDA address with dust lamports , causing the account creation to fail since:

1. The PDA address is deterministic: `["extra-account-metas", mint]`
2. An attacker can calculate it in advance
3. Transfer lamports to that address before the legitimate initialize call
4. `create_account` will fail because the account has non zero lamports

Vulnerable Code

```rust
// PDA is deterministic and predictable
let mint_key = mint.key();
let signer_seeds: &[&[u8]] = &[
   EXTRA_ACCOUNT_METAS_SEED,
   mint_key.as_ref(),
];
let (_, bump) = Pubkey::find_program_address(signer_seeds, &crate::ID);


// Attacker can pre-fund this address
system_program::create_account(
   CpiContext::new_with_signer(
       ctx.accounts.system_program.to_account_info(),
       system_program::CreateAccount {
           from: ctx.accounts.authority.to_account_info(),
           to: extra_account_metas.to_account_info(),  // Attacker pre-funded
       },
       &[signer_seeds_with_bump],
   ),
   lamports,
   account_size as u64,
   &crate::ID,
)?;
```

Remediation

Check if the account already exists and use `realloc` or `assign`/`allocate`/`transfer` flow instead:

```rust
let extra_account_metas = &ctx.accounts.extra_account_metas;
let mint_key = mint.key();
let signer_seeds: &[&[u8]] = &[
   EXTRA_ACCOUNT_METAS_SEED,
   mint_key.as_ref(),
];
let (_, bump) = Pubkey::find_program_address(signer_seeds, &crate::ID);
let signer_seeds_with_bump: &[&[u8]] = &[
   EXTRA_ACCOUNT_METAS_SEED,
   mint_key.as_ref(),
   &[bump],
];


let account_size = ExtraAccountMetaList::size_of(account_metas.len())?;
let lamports = Rent::get()?.minimum_balance(account_size as usize);


// ✅ Check if account exists
if extra_account_metas.lamports() == 0 {
   // Account doesn't exist, create it normally
   system_program::create_account(
       CpiContext::new_with_signer(
           ctx.accounts.system_program.to_account_info(),
           system_program::CreateAccount {
               from: ctx.accounts.authority.to_account_info(),
               to: extra_account_metas.to_account_info(),
           },
           &[signer_seeds_with_bump],
       ),
       lamports,
       account_size as u64,
       &crate::ID,
   )?;
} else {
//use allocate, assign, transfer flow
}


ExtraAccountMetaList::init::<ExecuteInstruction>(
   &mut extra_account_metas.try_borrow_mut_data()?,
   &account_metas,
)?;
```

Batch whitelist function does not create accounts if they don’t exist, silently skipping uninitialized entries

Resolved

Path

lib.rs

Function Name

`add_batch_to_whitelist()`

Description

The `add_batch_to_whitelist()` function is supposed to add multiple wallets to the whitelist, but it only sets `is_active = true` if the account is already initialized. If an account doesn’t exist or isn’t properly initialized, the function silently continues without actually creating or activating it.

Vulnerable Code

```rust
pub fn add_batch_to_whitelist(
    ctx: Context<AddBatchToWhitelist>,
    wallets: Vec<Pubkey>
) -> Result<()> {
    let mint_key = ctx.accounts.mint.key();
    let remaining = &ctx.remaining_accounts;


    for (I, wallet) in wallets.iter().enumerate() {
        let account_info = &remaining[I];


        // Verify PDA
        let (expected_pda, _bump) = Pubkey::find_program_address(
            &[WHITELIST_ENTRY_SEED, mint_key.as_ref(), wallet.as_ref()],
            &crate::ID,
        );
        require!(
            account_info.key() == expected_pda,
            TransferHookError::InvalidWhitelistEntry
        );


        let mut data = account_info.try_borrow_mut_data()?;
        if data.len() >= WhitelistEntry::INIT_SPACE + 8 {
            let disc = &data[..8];
            let expected_disc = WhitelistEntry::DISCRIMINATOR;
            if disc == expected_disc {
                //  Already initialized, just activate
                data[8 + 32 + 32] = 1; // is_active = true
            }
        }
        // If data.len() is too small, NOTHING HAPPENS
    }


    Ok(())
}
```

Impact

Silent failures: Wallets appear to be added but aren’t actually whitelisted

Recommendation

If data.len() is zero, which means account doesnt exist, initialize it if intended on `add_batch_to_whitelist`

Low Severity Issues

Execute function does not verify token accounts are in transferring state

Resolved

Path

lib.rs

Description

The `execute()` function processes transfers without verifying that the token accounts are actually in a valid transfer state. Token-2022 accounts have additional flags that can restrict transfers. The token program always sets the transferring flag in all token accounts involved in the transfer to true. This simple check ensures that a threat actor doesn't call your transfer hook outside of a transfer. The hook should verify that both source and destination accounts allow transfers before processing the whitelist check.

Vulnerable Code

```rust
pub fn execute(ctx: Context<Execute>, _amount: u64) -> Result<()> {
    let source_whitelist = &ctx.accounts.source_whitelist_entry;
    let destination_whitelist = &ctx.accounts.destination_whitelist_entry;


    // No check for account transfer state/frozen status
    require!(
        source_whitelist.is_active,
        TransferHookError::SenderNotAllowed
    );
    require!(
        destination_whitelist.is_active,
        TransferHookError::ReceiverNotAllowed
    );


    Ok(())
}
```

Recommendation

```rust
fn assert_is_transferring(ctx: &Context<TransferHook>) -> Result<()> {
    let source_token_info = ctx.accounts.source_token.to_account_info();
    let source_account_data_ref: Ref<&mut [u8]> =
source_token_info.try_borrow_data()?;
    let source_account =
PodStateWithExtensions::<PodAccount>::unpack(*source_account_data_ref)?;
    let source_account_extension =
source_account.get_extension::<TransferHookAccount>()?;
```

```
    let destination_token_info =
ctx.accounts.destination_token.to_account_info();
    let destination_account_data_ref: Ref<&mut [u8]> =
destination_token_info.try_borrow_data()?;
    let destination_account =
PodStateWithExtensions::<PodAccount>::unpack(*destination_account_data_ref)?;
    let destination_account_extension =
destination_account.get_extension::<TransferHookAccount>()?;


    if !bool::from(source_account_extension.transferring)
        || !bool::from(destination_account_extension.transferring)
    {
        return err!(TransferError::IsNotCurrentlyTransferring);
    }


    Ok(())
}
```

Closing Summary

In this report, we have considered the security of Atomyx Contract. We performed our audit according to the procedure described above.

2 issues of Critical severity, 3 issues of Medium severity & 1 issue of low severity were found. The Atomyx Team resolved all the issues mentioned.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Ouraudit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

May 2026

For

atomyx

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com